

NumPy Masterclass - Chapter 1

Introduction to NumPy & Array Basics

1. Import NumPy

Syntax:

```
import numpy as np
```

2. Create Arrays

1D:

```
np.array([1,2,3])
```

2D:

```
np.array([[1,2],[3,4]])
```

3. Why NumPy?

Python lists are flexible but slower for numerical work. NumPy arrays are faster and support vectorized operations.

4. Array Properties

a.shape -> rows, columns

a.ndim -> dimensions

a.size -> total elements

a.dtype -> data type

5. Example

```
a=np.array([[1,2,3],[4,5,6]])
```

```
a.shape = (2,3)
```

```
a.ndim = 2
```

```
a.size = 6
```

Practice

Create a 3x3 array, print its shape, size, ndim and dtype.

NumPy Masterclass - Chapter 2

Creating Arrays in NumPy

In this chapter you'll learn every common way to create arrays with short explanations and examples.

1. `np.array()`

Creates an array from a Python list.

Code

```
a=np.array([10,20,30])  
print(a)
```

Output

```
[10 20 30]
```

2. 2D Array

```
a=np.array([[1,2,3],[4,5,6]])  
Rows=2, Columns=3
```

3. `np.zeros()`

```
np.zeros((2,3))
```

Output:

```
[[0. 0. 0.]  
 [0. 0. 0.]]
```

4. `np.ones()`

`np.ones((3,2))` gives every element as 1.

5. `np.full()`

```
np.full((2,2),7)
```

Output:

```
[[7 7]  
 [7 7]]
```

6. `np.eye()`

Identity Matrix

```
np.eye(3)
```

Output:

```
[[1 0 0]  
 [0 1 0]  
 [0 0 1]]
```

7. np.arange()

`np.arange(1,11,2)`

Output: [1 3 5 7 9]

Syntax: `np.arange(start, stop, step)`

8. np.linspace()

`np.linspace(0,10,5)`

Output: [0. 2.5 5. 7.5 10.]

Creates equally spaced values.

9. Difference

arange() uses a step size.

linspace() uses number of values.

10. Quick Practice

1. Create a 4×4 zero matrix.
2. Create a 3×5 matrix of ones.
3. Create numbers from 5 to 50 with step 5.
4. Create 8 equally spaced numbers from 0 to 1.

Quick Revision Table

Function	Purpose
<code>np.array()</code>	Create array from list
<code>np.zeros()</code>	All zeros
<code>np.ones()</code>	All ones
<code>np.full()</code>	Fill with one value
<code>np.eye()</code>	Identity matrix
<code>np.arange()</code>	Range with step
<code>np.linspace()</code>	Equal intervals

NumPy Masterclass - Chapter 3

Array Properties (shape, ndim, size, dtype)

This chapter explains how to inspect a NumPy array using practical examples instead of lengthy theory.

Example Array

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
print(a)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

Property	Meaning	Example	Output
shape	Rows and Columns	a.shape	(2, 3)
ndim	Number of dimensions	a.ndim	2
size	Total elements	a.size	6
dtype	Data type of elements	a.dtype	int64 (or int32 depending on system)

Detailed Example

```
a = np.array([[10,20,30],[40,50,60]])
print(a.shape) # (2,3)
print(a.ndim) # 2
print(a.size) # 6
print(a.dtype) # int64
```

Explanation:

- shape tells us the array has 2 rows and 3 columns.
- ndim tells us this is a 2-D array.
- size counts every element.
- dtype tells the type stored inside the array.

Changing Data Type

```
b = a.astype(float)
print(b.dtype)
Output: float64
```

Practice Questions

1. Create a 3×4 array and print its shape.
2. Create a 1D array of 10 elements and print ndim.
3. Find the size of a 4×5 matrix.
4. Convert an integer array into float using astype().

Common Mistakes

- Confusing **size** with **shape**.
- Thinking `ndim` means number of rows—it means number of dimensions.
- Forgetting that `dtype` depends on the data stored.

NumPy Masterclass - Chapter 4

Indexing & Slicing

Learn how to access and modify elements in NumPy arrays using indexing and slicing. Every concept is explained through examples.

1. 1D Indexing

```
a=np.array([10,20,30,40,50])
print(a[0])    #10
print(a[2])    #30
print(a[-1])   #50 (last element)
Negative indexing starts from the end.
```

2. 2D Indexing

```
a=np.array([[1,2,3],[4,5,6],[7,8,9]])
print(a[1,2])  #6
print(a[2,0])  #7
Syntax: a[row, column]
```

Slice	Meaning	Example	Output
a[1:4]	Elements 1 to 3	[20,30,40]	20 30 40
a[:3]	First 3 elements		10 20 30
a[2:]	From index 2		30 40 50
a[::2]	Step = 2		10 30 50
a[::-1]	Reverse		50 40 30 20 10

3. 2D Slicing

```
a=np.array([[1,2,3],
            [4,5,6],
            [7,8,9]])

a[0,:] # First row -> [1 2 3]
a[:,1] # Second column -> [2 5 8]
a[0:2,:] # First two rows
a[:,0:2] # First two columns
a[1:,1:] # Bottom-right block
```

4. Modifying Values

```
a=np.array([10,20,30,40])
a[1]=99
# [10 99 30 40]

a[1:3]=0
# [10 0 0 40]
```

5. Slicing in Cramer's Rule

```
A=A.copy()
```

```
A[:,0]=B
```

: selects all rows.

0 selects the first column.

The entire first column is replaced by vector **B**.

Common Mistakes

- End index is **excluded** in slicing.
- Mixing row and column order.
- Forgetting that negative indices count from the end.

Practice

1. Print the last element using negative indexing.
2. Print the third column of a 3×3 array.
3. Reverse an array using slicing.
4. Replace the second column of a matrix with another array using slicing.

NumPy Masterclass - Chapter 5

Reshape, Flatten & Resize

Learn how to change the shape of arrays without changing their data.

1. reshape()

Purpose: Changes the shape of an array while keeping the same elements.

```
import numpy as np
a=np.arange(1,13)
print(a)
b=a.reshape(3,4)
print(b)
```

Output:

1D array →

```
[1 2 3 4 5 6 7 8 9 10 11 12]
```

Reshaped (3×4):

```
[[1 2 3 4]
 [5 6 7 8]
 [9 10 11 12]]
```

Rule	Explanation
Total elements must remain same	12 elements → (3×4), (2×6), (4×3) are valid
Invalid reshape	12 elements cannot become (5×3) because 15≠12
Use -1	NumPy calculates one dimension automatically

2. reshape(-1)

```
a=np.arange(12)
print(a.reshape(2, -1))
```

Output:

```
[[0 1 2 3 4 5]
 [6 7 8 9 10 11]]
```

NumPy automatically finds the missing dimension.

3. flatten()

Purpose: Converts a multi-dimensional array into a 1D array.

```
a=np.array([[1,2],[3,4]])
print(a.flatten())
```

Output: [1 2 3 4]

flatten() returns a **copy** of the array.

4. ravel()


```
a=np.array([[1,2],[3,4]])  
print(a.ravel())  
Output: [1 2 3 4]  
ravel() usually returns a view, making it more memory efficient.
```

5. **resize()**

```
a=np.array([1,2,3,4])  
a.resize((2,2))  
print(a)  
Output:  
[[1 2]  
 [3 4]]  
resize() modifies the original array.
```

Comparison

reshape() → Changes shape, original data unchanged.
flatten() → Converts to 1D copy.
ravel() → Converts to 1D view when possible.
resize() → Changes original array itself.

Practice Questions

1. Convert a 1D array of 16 elements into a 4×4 matrix.
2. Use reshape(-1) on a 24-element array.
3. Convert a 3×3 matrix into a 1D array using flatten().
4. Try an invalid reshape and observe the error.

NumPy Masterclass - Chapter 6

Mathematical Operations

Learn how NumPy performs element-wise mathematical operations quickly and efficiently.

Example Arrays

```
import numpy as np
a=np.array([10,20,30])
b=np.array([1,2,3])
```

Operation	Code	Output
Addition	a+b	[11 22 33]
Subtraction	a-b	[9 18 27]
Multiplication	a*b	[10 40 90]
Division	a/b	[10. 10. 10.]
Power	a**2	[100 400 900]
Modulus	a%b	[0 0 0]

Scalar Operations

```
a=np.array([1,2,3])
print(a+5) # [6 7 8]
print(a*10) # [10 20 30]
print(a/2) # [0.5 1.0 1.5]
```

A single number is automatically applied to every element (broadcasting).

Useful Mathematical Functions

```
np.sqrt(a) # Square root
np.square(a) # Square
np.abs([-3,2,-5]) # Absolute value
np.exp(a) # ex
np.log(a) # Natural log
np.sin(a), np.cos(a), np.tan(a)
```

Maximum, Minimum & Sum

```
a=np.array([4,8,2,10])
np.max(a) #10
np.min(a) #2
np.sum(a) #24
np.prod(a) #640
```

2D Array Operations

```
A=np.array([[1,2],[3,4]])  
B=np.array([[5,6],[7,8]])
```

```
A+B
```

```
A-B
```

```
A*B # Element-wise multiplication
```

For matrix multiplication, use **A@B** or **np.dot(A,B)** (covered in Matrix chapter).

Common Mistakes

- `*` performs element-wise multiplication, not matrix multiplication.
- Arrays should have compatible shapes for element-wise operations.
- Division returns floating-point values.

Practice

1. Add two arrays of five elements.
2. Find the square root of an array.
3. Multiply an array by 100.
4. Find max, min and sum of a 2D array.

NumPy Masterclass - Chapter 7

Universal Functions (Ufuncs)

Universal Functions (ufuncs) are fast NumPy functions that operate element-by-element on arrays without using loops.

1. What is a Ufunc?

Instead of writing loops, NumPy performs calculations on every element automatically.

```
import numpy as np
a=np.array([1,2,3,4])
print(np.sqrt(a))
Output: [1. 1.414 1.732 2.]
```

Function	Purpose	Example
np.sqrt()	Square Root	np.sqrt(a)
np.square()	Square	np.square(a)
np.abs()	Absolute Value	np.abs([-2,3,-4])
np.exp()	e ^x	np.exp(a)
np.log()	Natural Log	np.log(a)
np.sin()	Sine	np.sin(a)
np.cos()	Cosine	np.cos(a)
np.tan()	Tangent	np.tan(a)

2. Rounding Functions

```
a=np.array([1.2,2.7,3.5])
np.floor(a) # [1. 2. 3.]
np.ceil(a) # [2. 3. 4.]
np.round(a) # [1. 3. 4.]
```

3. Comparison Ufuncs

```
a=np.array([10,20,30])
b=np.array([15,20,25])
np.maximum(a,b) # [15 20 30]
np.minimum(a,b) # [10 20 25]
```

4. Logical Ufuncs

```
a=np.array([1,2,3,4])
a>2
```

Output: [False False True True]
Useful for filtering data in Machine Learning.

5. Why Ufuncs are Important

- Much faster than Python loops.
- Less code to write.
- Optimized in C internally.
- Used heavily in NumPy, Pandas, and Machine Learning libraries.

Common Mistakes

- Using Python's **math** module on arrays instead of NumPy functions.
- Applying **np.log()** to zero or negative values.
- Forgetting that trigonometric functions use radians.

Practice Questions

1. Find square roots of numbers from 1 to 10.
2. Round an array using floor, ceil, and round.
3. Compare two arrays using maximum() and minimum().
4. Create a boolean array using $a > 5$.

NumPy Masterclass - Chapter 8

Statistical Functions

Statistical functions help summarize data. They are heavily used in data analysis and machine learning.

Example Array

```
import numpy as np
a=np.array([10,20,30,40,50])
```

Function	Purpose	Example	Output
np.mean(a)	Average	np.mean(a)	30
np.median(a)	Middle value	np.median(a)	30
np.sum(a)	Total sum	np.sum(a)	150
np.min(a)	Smallest value	np.min(a)	10
np.max(a)	Largest value	np.max(a)	50
np.std(a)	Standard deviation	np.std(a)	14.14
np.var(a)	Variance	np.var(a)	200
np.prod(a)	Product	np.prod(a)	12000000

Axis in 2D Arrays

```
A=np.array([[1,2,3],
            [4,5,6]])
```

```
np.sum(A) #21
np.sum(A,axis=0) #[5 7 9] Column-wise
np.sum(A,axis=1) #[6 15] Row-wise
```

axis=0 → operate column-wise.

axis=1 → operate row-wise.

Finding Mean Row-wise and Column-wise

```
np.mean(A,axis=0)
np.mean(A,axis=1)
```

Machine Learning Use

- Mean is used for missing value imputation.
- Standard deviation is used in feature scaling (standardization).
- Min and Max are used in normalization.
- Sum and Product are used in numerical computations.

Common Mistakes

- Confusing **mean** with **median**.
- Forgetting the **axis** parameter for 2D arrays.
- Assuming `std()` uses sample standard deviation—it computes population standard deviation by default.

Practice Questions

1. Find mean and median of an array.
2. Calculate row-wise and column-wise sums.
3. Find standard deviation and variance.
4. Find min, max and product of a matrix.

NumPy Masterclass - Chapter 9

Searching, Sorting & Filtering Arrays

These functions help locate, arrange, and filter data efficiently. They are frequently used in data preprocessing and machine learning.

Example Array

```
import numpy as np
a=np.array([10,5,30,5,40,20])
```

Function	Purpose	Example	Output
np.sort(a)	Sort array	np.sort(a)	[5 5 10 20 30 40]
np.argsort(a)	Sorted indices	np.argsort(a)	[1 3 0 5 2 4]
np.where(a==5)	Find matching values	np.where(a==5)	(array([1,3]),)
np.argmax(a)	Index of largest value	np.argmax(a)	4
np.argmin(a)	Index of smallest value	np.argmin(a)	1
np.unique(a)	Remove duplicates	np.unique(a)	[5 10 20 30 40]

Boolean Filtering

```
a=np.array([10,5,30,5,40,20])
print(a[a>20])
```

Output: [30 40]

The expression **a>20** creates a boolean array, and NumPy returns only the elements where the condition is True.

Multiple Conditions

```
print(a[(a>=10) & (a<40)])
Output: [10 30 20]
```

Use **&** for AND and **|** for OR. Always wrap each condition in parentheses.

Frequency Counting

```
b=np.array([1,1,2,2,2,3,4,4])
print(np.bincount(b))
Output: [0 2 3 1 2]
```

The value at each index represents the count of that number.

Machine Learning Uses

- Remove unwanted values before training.
- Detect duplicates using **unique()**.
- Find highest/lowest values using **argmax()** and **argmin()**.

- Filter rows based on conditions while cleaning datasets.

Common Mistakes

- Using Python's **and** instead of **&** for arrays.
- Forgetting parentheses around multiple conditions.
- Confusing **sort()** with **argsort()**.

Practice Questions

1. Sort an array in ascending order.
2. Find the indices where the value is 10.
3. Extract all elements greater than 25.
4. Find unique values from an array with duplicates.
5. Find the index of the largest and smallest values.

NumPy Masterclass - Chapter 10

Broadcasting

Broadcasting allows NumPy to perform arithmetic on arrays of different shapes without writing loops.

1. What is Broadcasting?

Instead of copying values manually, NumPy automatically expands the smaller array (virtually) so calculations can be performed element by element.

2. Scalar Broadcasting

```
import numpy as np
a=np.array([1,2,3,4])
print(a+10)
Output: [11 12 13 14]
```

The value **10** is applied to every element automatically.

3. Broadcasting with 1D Arrays

```
a=np.array([10,20,30])
b=np.array([1,2,3])
print(a+b)
Output: [11 22 33]
```

4. Broadcasting with 2D Arrays

```
A=np.array([[1,2,3],
            [4,5,6]])
B=np.array([10,20,30])

print(A+B)
Output:
[[11 22 33]
 [14 25 36]]
```

The 1D array is broadcast to each row of the matrix.

Shape A	Shape B	Compatible?	Reason
(3,)	Scalar	Yes	Scalar expands everywhere
(2,3)	(3,)	Yes	Matches last dimension
(2,3)	(2,1)	Yes	Single column expands
(2,3)	(2,)	No	Last dimensions don't match
(2,3)	(4,)	No	3 and 4 are incompatible

5. Broadcasting Rules

1. Compare dimensions from right to left.
2. Dimensions are compatible if they are equal OR one of them is 1.
3. Otherwise NumPy raises a broadcasting error.

6. Common Error

```
A=np.ones((2,3))
```

```
B=np.array([1,2])
```

```
A+B
```

Error: operands could not be broadcast together because shapes (2,3) and (2,) are incompatible.

Machine Learning Uses

- Feature scaling using constants.
- Adding bias vectors to matrices.
- Normalizing datasets.
- Fast preprocessing without Python loops.

Practice

1. Add 100 to every element of an array.
2. Add a row vector to a 2×3 matrix.
3. Try broadcasting with shapes (3,1) and (3,).
4. Identify whether (4,5) and (5,) are compatible.

NumPy Masterclass - Chapter 11

Matrix Operations

Matrices are the backbone of machine learning. This chapter explains common matrix operations with concise examples.

Example Matrices

```
import numpy as np
A=np.array([[1,2],[3,4]])
B=np.array([[5,6],[7,8]])
```

Operation	Code	Output
Addition	A+B	[[6 8],[10 12]]
Subtraction	A-B	[[-4 -4],[-4 -4]]
Element-wise Multiplication	A*B	[[5 12],[21 32]]
Matrix Multiplication	A@B	[[19 22],[43 50]]
Dot Product	np.dot(A,B)	[[19 22],[43 50]]
Transpose	A.T	[[1 3],[2 4]]

Element-wise vs Matrix Multiplication

A * B multiplies corresponding elements.

```
[[1×5, 2×6],
 [3×7, 4×8]]
```

A @ B (or **np.dot(A,B)**) performs row × column multiplication following matrix multiplication rules.

Identity Matrix

```
I=np.eye(3)
print(I)
```

Output:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Multiplying any compatible matrix by an identity matrix returns the original matrix.

Matrix Dimensions

For matrix multiplication:

(m × n) @ (n × p) = (m × p)

Example:

(2×3) @ (3×4) → Result is (2×4).

Machine Learning Applications

- Input data is stored as matrices.
- Neural networks use matrix multiplication in every layer.
- Image processing treats images as matrices.
- Linear regression and deep learning rely heavily on these operations.

Common Mistakes

- Using `*` instead of `@` for matrix multiplication.
- Multiplying matrices with incompatible dimensions.
- Forgetting that transpose swaps rows and columns.

Practice Questions

1. Add two 3×3 matrices.
2. Multiply two compatible matrices using `@`.
3. Find the transpose of a matrix.
4. Create a 4×4 identity matrix using `np.eye()`.
5. Verify that $A @ I = A$.

NumPy Masterclass - Chapter 12

Linear Algebra with NumPy

Linear algebra is the mathematical foundation of Machine Learning. NumPy provides powerful functions through the `numpy.linalg` module.

Import Module

```
import numpy as np
from numpy import linalg
```

Function	Purpose	Example
<code>linalg.det(A)</code>	Determinant	<code>linalg.det(A)</code>
<code>linalg.inv(A)</code>	Inverse Matrix	<code>linalg.inv(A)</code>
<code>linalg.solve(A,B)</code>	Solve $AX=B$	<code>linalg.solve(A,B)</code>
<code>linalg.eig(A)</code>	Eigenvalues & Eigenvectors	<code>linalg.eig(A)</code>
<code>linalg.matrix_rank(A)</code>	Rank of Matrix	<code>linalg.matrix_rank(A)</code>

1. Determinant

```
A=np.array([[1,2],[3,4]])
print(np.linalg.det(A))
Output: -2.0
A matrix with determinant 0 has no inverse.
```

2. Inverse

```
A=np.array([[1,2],[3,4]])
print(np.linalg.inv(A))
The inverse exists only if determinant  $\neq 0$ .
```

3. Solving Linear Equations

```
Solve:
2x+y=5
x+3y=6

A=np.array([[2,1],[1,3]])
B=np.array([5,6])
print(np.linalg.solve(A,B))
Output: [1.8 1.4]
```

4. Eigenvalues & Eigenvectors

```
A=np.array([[4,2],[1,3]])
values,vectors=np.linalg.eig(A)
Eigenvalues measure important directions in data. They are widely used in PCA and dimensionality
```

reduction.

5. Matrix Rank

```
A=np.array([[1,2],[2,4]])  
print(np.linalg.matrix_rank(A))
```

Output: 1

Rank tells how many independent rows or columns a matrix has.

Machine Learning Connection

- Determinant checks matrix invertibility.
- Inverse is used in Linear Regression equations.
- Eigenvalues/Eigenvectors are used in PCA.
- Solving linear systems appears in optimization and regression.

Common Mistakes

- Calling `inv()` on a singular matrix (determinant = 0).
- Using `solve()` with incompatible dimensions.
- Confusing element-wise division with matrix inverse.

Practice

1. Find the determinant of a 3×3 matrix.
2. Find the inverse of a 2×2 matrix.
3. Solve $AX=B$ using `linalg.solve()`.
4. Find eigenvalues of a matrix.
5. Compute the rank of a matrix.

NumPy Masterclass - Chapter 13

Random Module (numpy.random)

The `numpy.random` module generates random numbers for simulations, testing, and machine learning datasets.

Function	Purpose	Example
<code>np.random.rand()</code>	Uniform random decimals (0-1)	<code>np.random.rand(2,3)</code>
<code>np.random.randn()</code>	Normally distributed values	<code>np.random.randn(3)</code>
<code>np.random.randint()</code>	Random integers	<code>np.random.randint(1,10,5)</code>
<code>np.random.choice()</code>	Random selection	<code>np.random.choice([10,20,30],2)</code>
<code>np.random.shuffle()</code>	Shuffle original array	<code>np.random.shuffle(a)</code>
<code>np.random.permutation()</code>	Return shuffled copy	<code>np.random.permutation(a)</code>
<code>np.random.seed()</code>	Reproducible results	<code>np.random.seed(42)</code>

1. Random Decimal Numbers

```
import numpy as np
np.random.rand(2,3)
```

Example Output:

```
[[0.54 0.71 0.60]
 [0.54 0.42 0.64]]
```

2. Random Integers

```
np.random.randint(1,100,5)
```

Returns five random integers between 1 and 99.

3. Random Choice

```
colors=np.array(["Red","Blue","Green"])
np.random.choice(colors,2)
```

Randomly selects elements from an array.

4. Shuffle vs Permutation

```
a=np.array([1,2,3,4,5])
np.random.shuffle(a)
```

shuffle() changes the original array.

```
b=np.random.permutation(a)
```

permutation() returns a shuffled copy while leaving the original unchanged.

5. Random Seed


```
np.random.seed(42)
print(np.random.randint(1, 10, 5))
```

Using the same seed always produces the same random sequence, making experiments reproducible.

Machine Learning Uses

- Split datasets randomly.
- Initialize neural network weights.
- Generate synthetic datasets.
- Shuffle training data before each epoch.

Common Mistakes

- Forgetting to set a seed when reproducible results are required.
- Confusing **shuffle()** (modifies original array) with **permutation()** (returns a copy).

Practice Questions

1. Generate a 3×3 matrix of random decimals.
2. Generate ten random integers between 50 and 100.
3. Randomly choose three values from an array.
4. Compare `shuffle()` and `permutation()`.
5. Use `seed(100)` and verify identical outputs across runs.

NumPy Masterclass - Chapter 14

Copy vs View

Understanding the difference between `copy()` and `view()` is important because it affects memory usage and whether changes are reflected in the original array.

Method	Memory	Changes Original?	Changes Copy/View?
<code>copy()</code>	New memory	No	No
<code>view()</code>	Shared memory	Yes	Yes

1. Using `copy()`

```
import numpy as np
a=np.array([10,20,30])
b=a.copy()
b[0]=99
print(a) # [10 20 30]
print(b) # [99 20 30]
```

Explanation: `copy()` creates a completely new array. Changes in **b** do not affect **a**.

2. Using `view()`

```
a=np.array([10,20,30])
b=a.view()
b[0]=99
print(a) # [99 20 30]
print(b) # [99 20 30]
```

Explanation: `view()` shares the same memory. Modifying one array also modifies the other.

3. Checking Memory Ownership

```
a.base
b.base
```

If **base** is **None**, the array owns its data. A view usually has a non-None **base** pointing to the original array.

4. Why It Matters

- **`copy()`** is safer because accidental changes don't affect the original data.
- **`view()`** is faster and uses less memory because data is shared.

Machine Learning Uses

- Use **`copy()`** before preprocessing when you want to preserve the original dataset.
- Use **`view()`** when working with large arrays to save memory if shared data is acceptable.

Common Mistakes

- Assuming `view()` creates a new array—it does not.
- Accidentally modifying the original array through a view.
- Using `copy()` unnecessarily on very large datasets, increasing memory usage.

Practice Questions

1. Create an array and make a copy using `copy()`.
2. Create a view and modify an element. Observe both arrays.
3. Check the **base** attribute for a copy and a view.
4. Decide whether `copy()` or `view()` is better for different scenarios.

NumPy Masterclass - Chapter 15

Boolean & Fancy Indexing

Boolean and Fancy Indexing allow you to select, filter, and modify specific elements efficiently. These techniques are heavily used in Machine Learning data preprocessing.

1. Boolean Indexing

```
import numpy as np
a=np.array([10,25,30,15,50])
```

```
print(a[a>20])
```

Output: [25 30 50]

The condition creates a boolean array and NumPy returns only the matching elements.

2. Multiple Conditions

```
print(a[(a>=20) & (a<40)])
```

Output: [25 30]

Use:

& → AND

| → OR

~ → NOT

Always wrap each condition in parentheses.

Operator	Meaning	Example
&	AND	(a>10) & (a<40)
	OR	(a<15) (a>40)
~	NOT	~(a>20)

3. Fancy Indexing

```
a=np.array([10,20,30,40,50])
```

```
print(a[[0,2,4]])
```

Output: [10 30 50]

Pass a list of indices to select multiple elements.

4. Fancy Indexing in 2D Arrays

```
A=np.array([[1,2,3],
[4,5,6],
[7,8,9]])
```

```
print(A[[0,2]]) # First and third rows
```

```
print(A[:,[0,2]]) # First and third columns
```

5. Conditional Assignment

```
a=np.array([10,25,30,15,50])
a[a<20]=0
print(a)
```

Output: [10 becomes 0? Actually result: [0 25 30 0 50]]
All elements less than 20 are replaced with 0.

Machine Learning Uses

- Remove invalid values from datasets.
- Filter rows based on conditions.
- Replace missing or outlier values.
- Select important features before training models.

Common Mistakes

- Using Python's **and** or **or** instead of **&** and **|**.
- Forgetting parentheses around conditions.
- Mixing fancy indexing with slices incorrectly.

Practice Questions

1. Print all values greater than 100.
2. Select indices [1,3,5] from an array.
3. Replace all negative values with 0.
4. Select the first and last columns of a matrix using fancy indexing.
5. Use multiple conditions to filter values between 10 and 50.

NumPy Masterclass - Chapter 16

Cramer's Rule & Solving Linear Systems

Cramer's Rule solves a system of linear equations using determinants. NumPy makes the calculations simple using `numpy.linalg.det()`.

Problem Statement

Solve:

$$10x + 5y - 6z = 5$$

$$7x - 4y + 6z = -8$$

$$9x + 9y + z = 10$$

Step 1: Create Matrices

```
A=np.array([[10,5,-6],  
[7,-4,6],  
[9,9,1]])
```

```
B=np.array([5,-8,10])
```

Step 2: Determinant

```
D=np.linalg.det(A)
```

If **D = 0**, Cramer's Rule cannot be used because the system has no unique solution.

Step 3: Replace Columns Using Slicing

```
X=A.copy()  
X[:,0]=B    # Replace first column
```

```
Y=A.copy()  
Y[:,1]=B    # Replace second column
```

```
Z=A.copy()  
Z[:,2]=B    # Replace third column
```

: selects all rows, and **0/1/2** selects the required column.

Step 4: Find Dx, Dy and Dz

```
Dx=np.linalg.det(X)  
Dy=np.linalg.det(Y)  
Dz=np.linalg.det(Z)
```

Step 5: Calculate x, y and z

```
x=Dx/D  
y=Dy/D
```

$z = D_z / D$

```
print(x, y, z)
```

Formula	Meaning
$x = D_x / D$	Replace 1st column
$y = D_y / D$	Replace 2nd column
$z = D_z / D$	Replace 3rd column

Alternative Method

```
np.linalg.solve(A, B)
```

This directly computes the solution and is usually preferred for larger systems.

Common Mistakes

- Forgetting **A.copy()**, which modifies the original matrix.
- Replacing the wrong column.
- Applying Cramer's Rule when determinant is zero.
- Dividing by the wrong determinant.

Machine Learning Connection

Linear systems appear in linear regression, optimization, computer graphics and scientific computing. Understanding determinants and matrix equations builds a strong foundation for machine learning.

Practice

1. Solve another 3×3 system using Cramer's Rule.
2. Verify the answer using **np.linalg.solve()**.
3. Check what happens when determinant equals zero.

NumPy Masterclass - Chapter 17

Real Machine Learning Examples with NumPy

This chapter demonstrates how NumPy is used before training machine learning models. Most ML projects start by cleaning and preparing data with NumPy.

1. Creating a Dataset

```
import numpy as np
data=np.array([[22, 65], [25, 70], [30, 80], [35, 90]])
```

Each row represents one sample. Columns can represent features such as Age and Weight.

2. Selecting Features

```
X=data[:,0]
Y=data[:,1]
```

Use slicing to extract individual columns or features.

3. Feature Scaling (Normalization)

```
x=np.array([10, 20, 30, 40])
norm=(x-np.min(x))/(np.max(x)-np.min(x))
```

Output values are between 0 and 1, helping algorithms train more effectively.

4. Standardization

```
std=(x-np.mean(x))/np.std(x)
```

Standardization produces data with mean ≈ 0 and standard deviation ≈ 1 .

5. Handling Missing Values

```
a=np.array([10, np.nan, 20, 30])
mean=np.nanmean(a)
a[np.isnan(a)]=mean
```

Replace missing values with the mean before training a model.

6. Filtering Data

```
marks=np.array([40, 55, 90, 33, 78])
passed=marks[marks>=40]
```

Boolean indexing is widely used to filter records.

7. Shuffling Data

```
idx=np.random.permutation(len(data))
shuffled=data[idx]
```

Shuffling prevents models from learning patterns caused by data order.

Task	NumPy Function
Feature Selection	<code>[:, column]</code>
Normalization	<code>min(), max()</code>
Standardization	<code>mean(), std()</code>
Missing Values	<code>nanmean(), isnan()</code>
Filtering	Boolean Indexing
Shuffling	<code>random.permutation()</code>

Machine Learning Workflow

Collect Data → Convert to NumPy Array → Clean Missing Values → Filter Records → Normalize/Standardize Features → Shuffle Data → Train Machine Learning Model.

Mini Project

Given a dataset of student marks:

1. Find the average.
2. Replace missing values.
3. Normalize marks.
4. Select students scoring above 60.
5. Shuffle the final dataset.

Interview Tips

- Explain the difference between normalization and standardization.
- Know why shuffling is important.
- Be comfortable with slicing, boolean indexing, and statistical functions.

NumPy Practice Questions with Answers (Chapters 1–16)

Chapter 1

Q. Create a 1D array. A.

```
import numpy as np
a=np.array([1,2,3,4])
```

Chapter 2

Q. Create a 3×3 zero matrix.

A.

```
np.zeros((3,3))
```

Chapter 3

Q. Create a 3×4 array and print shape, size, ndim and dtype.

A.

```
a=np.arange(12).reshape(3,4)
print(a.shape)
print(a.size)
print(a.ndim)
print(a.dtype)
```

Chapter 4

Q. Reverse an array using slicing.

A.

```
a=np.array([1,2,3,4,5])
print(a[::-1])
```

Chapter 5

Q. Convert 16 elements into a 4×4 matrix.

A.

```
a=np.arange(1,17)
print(a.reshape(4,4))
```

Chapter 6

Q. Add two arrays.

A.

```
a=np.array([1,2,3])
b=np.array([4,5,6])
print(a+b)
```

Chapter 7

Q. Find square roots of 1–10.

A.

```
a=np.arange(1,11)
print(np.sqrt(a))
```

Chapter 8

Q. Find mean and standard deviation.

A.

```
a=np.array([10,20,30,40])
print(np.mean(a))
```

```
print(np.std(a))
```

Chapter 9

Q. Filter elements greater than 25.

A.

```
a=np.array([10,30,40,15])  
print(a[a>25])
```

Chapter 10

Q. Add 100 to every element.

A.

```
a=np.array([1,2,3])  
print(a+100)
```

Chapter 11

Q. Matrix multiplication.

A.

```
A=np.array([[1,2],[3,4]])  
B=np.array([[5,6],[7,8]])  
print(A@B)
```

Chapter 12

Q. Solve $AX=B$.

A.

```
A=np.array([[2,1],[1,3]])  
B=np.array([5,6])  
print(np.linalg.solve(A,B))
```

Chapter 13

Q. Generate five random integers.

A.

```
np.random.randint(1,100,5)
```

Chapter 14

Q. Demonstrate `copy()`.

A.

```
a=np.array([1,2,3])  
b=a.copy()  
b[0]=99  
print(a,b)
```

Chapter 15

Q. Replace negative values with 0.

A.

```
a=np.array([-1,2,-3,4])  
a[a<0]=0  
print(a)
```

Chapter 16

Q. Solve using Cramer's Rule.

A.

```
D=np.linalg.det(A)  
X=A.copy(); X[:,0]=B
```

```
Y=A.copy(); Y[:,1]=B
Z=A.copy(); Z[:,2]=B
x=np.linalg.det(X)/D
y=np.linalg.det(Y)/D
z=np.linalg.det(Z)/D
```

NumPy Masterclass - Chapter 19

Horizontal & Vertical Stacking + Cramer's Rule

Learn how to combine arrays using NumPy stacking functions and apply them to solve Cramer's Rule.

Vertical Stacking (`np.vstack()`)

```
import numpy as np
A=np.array([1,2,3])
B=np.array([4,5,6])
print(np.vstack((A,B)))
Output:
[[1 2 3]
 [4 5 6]]
```

Horizontal Stacking (`np.hstack()`)

```
A=np.array([1,2,3])
B=np.array([4,5,6])
print(np.hstack((A,B)))
Output:
[1 2 3 4 5 6]
```

Function	Purpose	Example
<code>np.vstack()</code>	Join row-wise	<code>np.vstack((A,B))</code>
<code>np.hstack()</code>	Join column-wise	<code>np.hstack((A,B))</code>
<code>np.column_stack()</code>	Make columns	<code>np.column_stack((A,B))</code>

Cramer's Rule using `hstack()`

```
A=np.array([[10,5,-6],
 [7,-4,6],
 [9,9,1]])
```

```
B=np.array([[5],[-8],[10]])
```

```
X=np.hstack((B,A[:,1:]))
Y=np.hstack((A[:,0:1],B,A[:,2:]))
Z=np.hstack((A[:, :2],B))
```

Each matrix replaces one column of A with B using horizontal stacking.

Find Solution

```
D=np.linalg.det(A)
Dx=np.linalg.det(X)
Dy=np.linalg.det(Y)
Dz=np.linalg.det(Z)
```

$x=Dx/D$

$y=Dy/D$

$z=Dz/D$

Practice Questions

1. Stack two arrays vertically.
2. Stack two arrays horizontally.
3. Create a matrix using `column_stack()`.
4. Solve a 3×3 system using `hstack()`.
5. Compare `hstack()` with slicing for Cramer's Rule.